

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

Лабораторная работа №2

по курсу **«Технологии разработки программного обеспечения»**

на тему

«Управление версиями и коллективной разработкой программного проекта на примере утилиты Git»

Выполнил: Солдаткин Н. С. _____ (подпись) _____ (дата)
Группа: ИУ5-11М

Проверил: Виноградова М. В. _____ (подпись) _____ (дата)

Москва, 2026 г.

1 Задание

1.1 Цель работы

Изучить возможности системы управления версиями и коллективной разработкой программного проекта; освоить методы работы с утилитой Git для управления версиями и коллективной разработкой программного проекта.

1.2 Средства выполнения

- утилита Git;
- утилита Git GUI и gitk.

1.3 Задачи

В ходе выполнения лабораторной работы были выполнены следующие пункты задания:

- запустить Git GUI и создать новый репозиторий в рабочей папке проекта;
- добавить в папку репозитория файл, зафиксировать состояние репозитория (выполнить commit);
- внести изменения в файл, зафиксировать новое состояние репозитория;
- создать новую ветку (lastname), внести в неё изменения и зафиксировать их;
- переключиться на ветку master, внести в неё изменения (исправление) и зафиксировать их;
- продемонстрировать слияние веток - разрешить возникший конфликт;
- просмотреть дерево изменений веток (историю), в том числе на примере выноса части кода в отдельные функцию и файлы;
- продемонстрировать откат изменений (revert);
- создать удалённый репозиторий на github.com и опубликовать в нём изменения.

2 Ход работы

2.1 Создание репозитория и фиксация начального состояния

С помощью контекстного меню проводника Windows в рабочей папке проекта `BMSTU_IU5_MAGA_TERM1-TRPO-Lab1` запущен Git GUI, создан новый локальный репозиторий. В каталог репозитория добавлен файл main.c с исходным кодом программы, выводящей приветствие «Hello world!». На рисунке 1 показано состояние main.c до добавления в список на фиксацию.

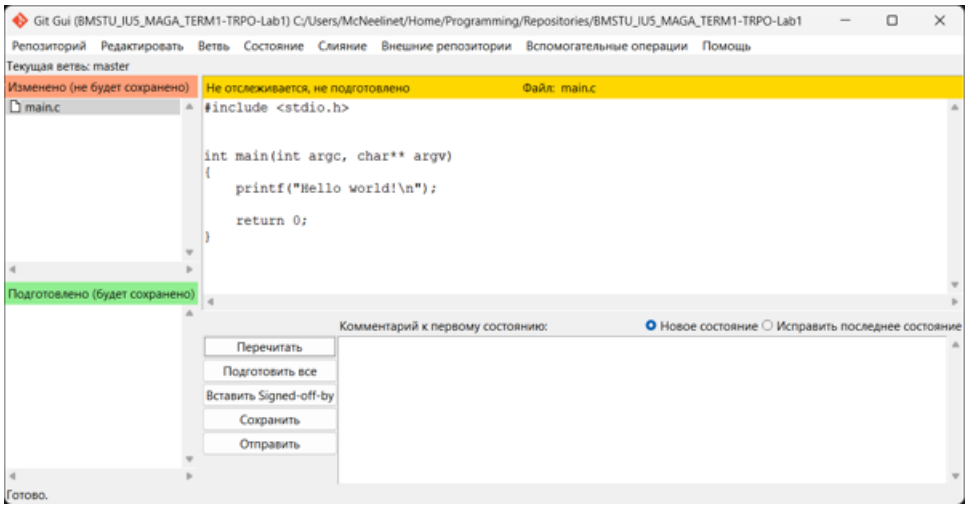


Рисунок 1 – Файл main.c перед добавлением в фиксацию (Git GUI)

После подготовки файла и ввода комментария к фиксации выполнен первый commit с сообщением «Initial commit». На рисунке 2 в окне gitk показана история репозитория и содержимое зафиксированного изменения.

2.2 Внесение изменений и повторная фиксация

Программа доработана: вместо статического текста main.c теперь запрашивает у пользователя имя (fgets) и выводит персонализированное приветствие. Изменения проиндексированы и зафиксированы отдельным коммитом «Give a personalized hello.». На рисунке 3 приведена фиксация данного изменения в gitk (показаны изменения в main.c).

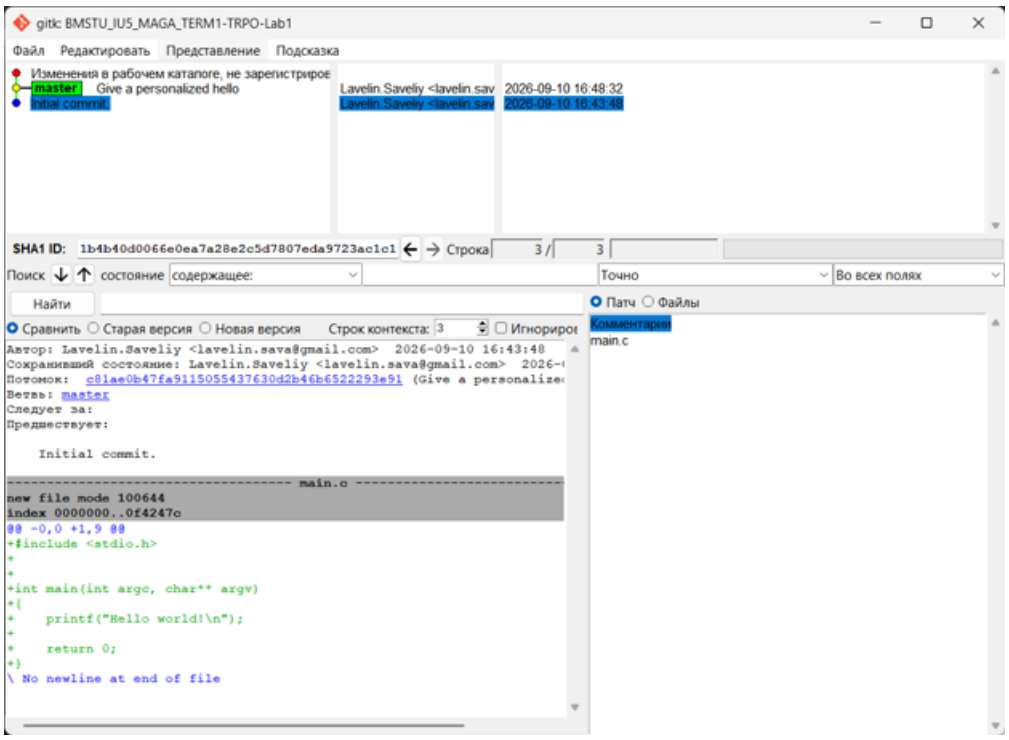


Рисунок 2 – История после фиксации начального состояния (Initial commit)

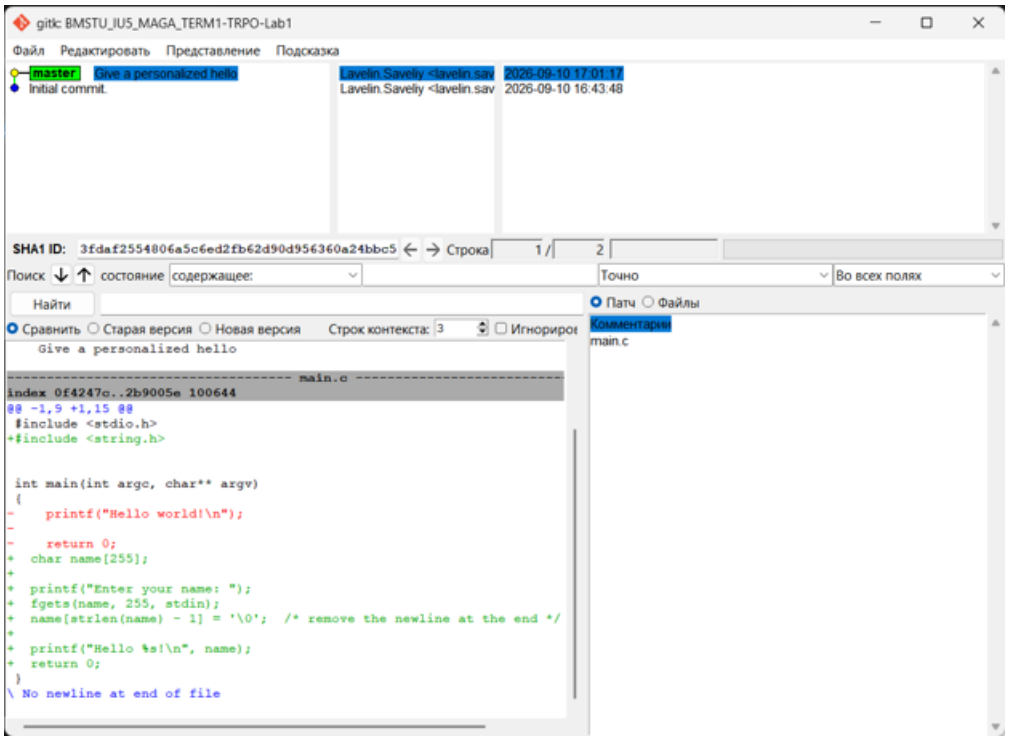


Рисунок 3 – Фиксация изменения «Give a personalized hello»

2.3 Ветвление

Для разработки нового функционала без риска для стабильной ветки создана и выбрана новая ветка «lastname». В этой ветке main.c изменён так, что-

бы программа запрашивала также фамилию пользователя и выводила её вместе с именем. Изменение зафиксировано коммитом «Prompt user for last name as well.». На рисунке 4 показана фиксация изменений в ветке lastname.

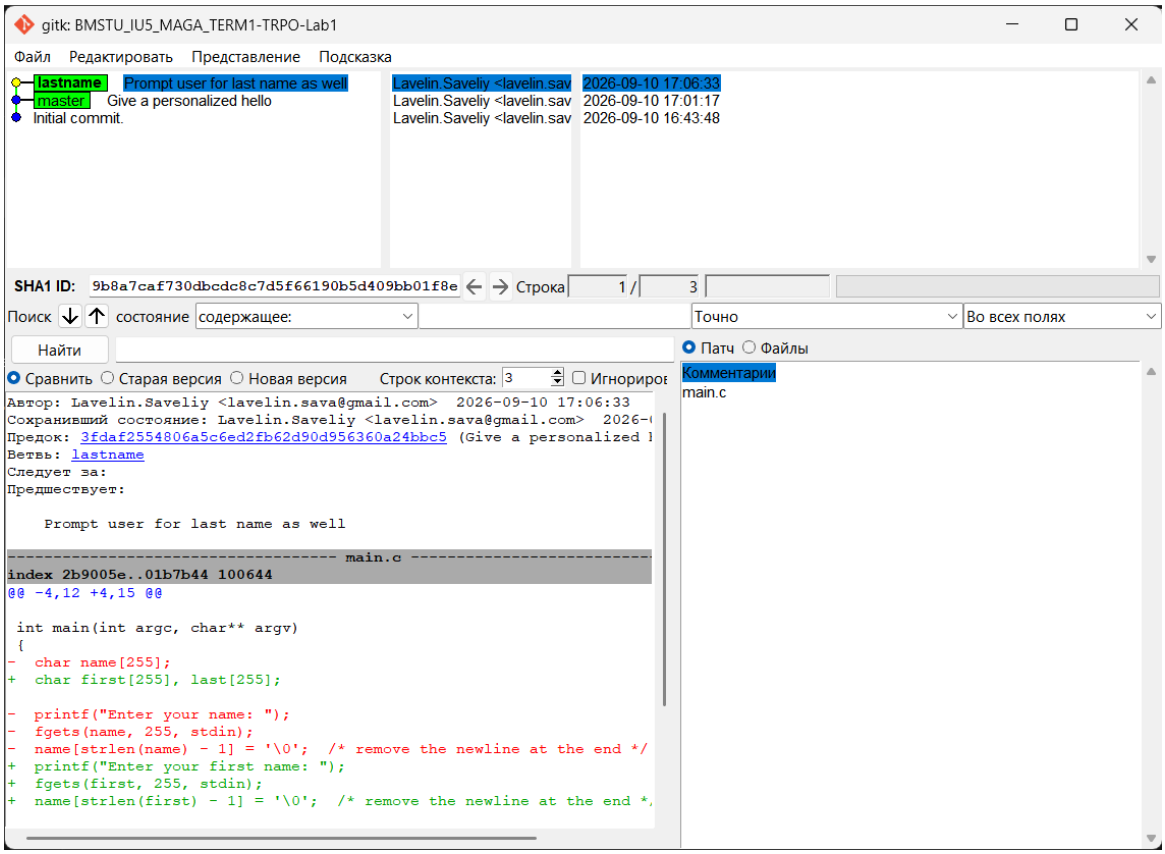


Рисунок 4 – Фиксация изменений в ветке lastname

Далее выполнено переключение обратно на ветку master. В ветке master исправлена грамматическая ошибка приветствия (добавлена запятая перед именем) и зафиксирован коммит «Use proper grammar.». На рисунке 5 приведён граф истории всех веток (Repository => Visualize All Branch History), демонстрирующий расхождение веток master и lastname от общего предка.

2.4 Слияние веток

Выполнено слияние ветки lastname в master. Так как обе ветки изменяли одну и ту же строку файла main.c, при слиянии возник конфликт. Конфликт разрешён вручную в текстовом редакторе Git GUI: в итоговом коде оставлены запрос имени и фамилии (из ветки lastname) вместе с исправленной грамматикой (из ветки master). После разрешения конфликта изменения проиндексированы и зафиксирован коммит слияния «Merge branch ’lastname’». На рисунке 6 показана фиксация слияния веток.

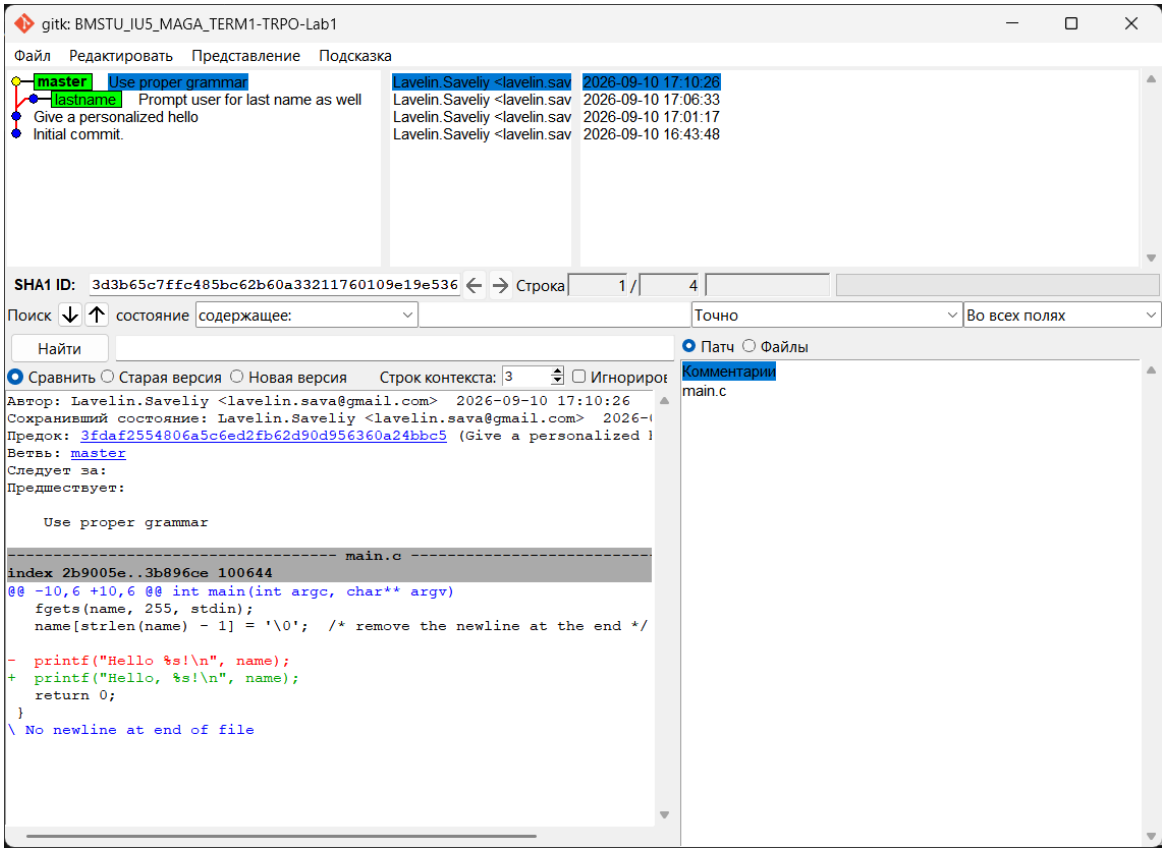


Рисунок 5 – Исправление в ветке master и история ветвления

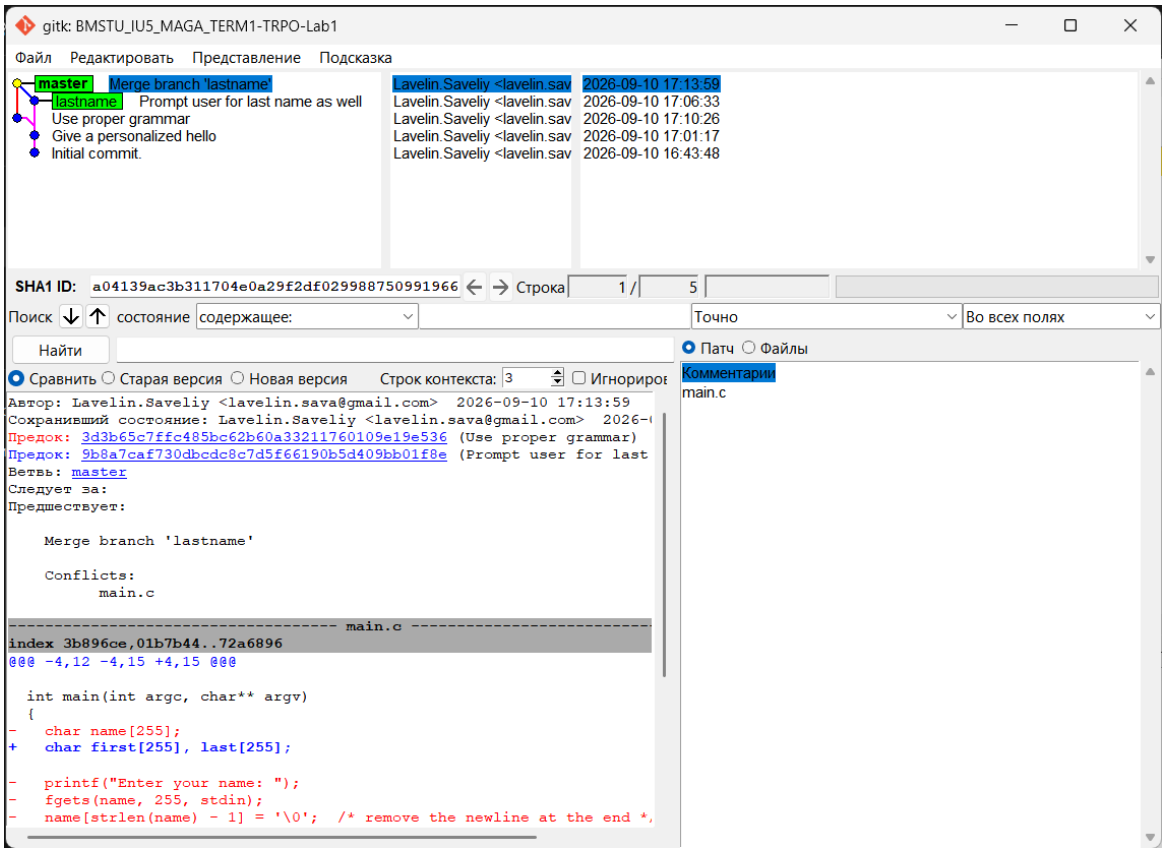


Рисунок 6 – Фиксация слияния ветки lastname в master с разрешением конфликта

2.5 Просмотр истории

Для демонстрации возможностей просмотра истории код запроса имени и фамилии вынесен из main.c в отдельную функцию askname() и отдельные файлы askname.c и askname.h. Изменение зафиксировано коммитом «Move the ask name code into its own function and file.». На рисунке 7 показана данная фиксация в gitk, а также использование истории для поиска изменений по нескольким файлам.

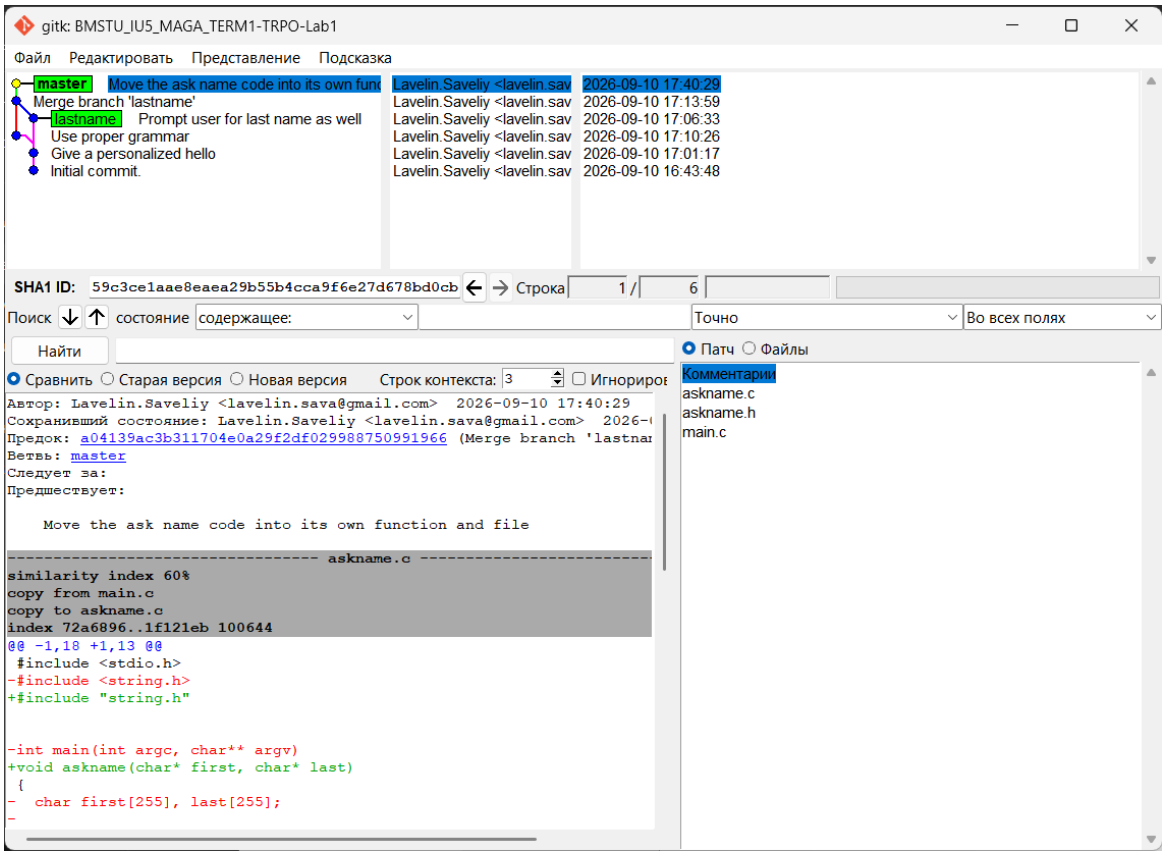


Рисунок 7 – Фиксация выноса кода в отдельные файлы askname.c и askname.h

Просмотр истории отдельного файла (в том числе через Repository => Browse master’s Files и git blame) позволяет установить автора и коммит, в котором была добавлена или перемещена конкретная строка кода. Это особенно полезно при совместной разработке несколькими участниками.

2.6 Отмена изменений (revert)

Продемонстрирован откат изменений через создание обратного коммита (revert): для коммита «Move the ask name code into its own function and file.» выполнена команда «Revert this commit». При этом создаётся новая фиксация,

отменяющая внесённые изменения, а сама история репозитория не переписывается. На рисунке 8 показан результат отката изменений.

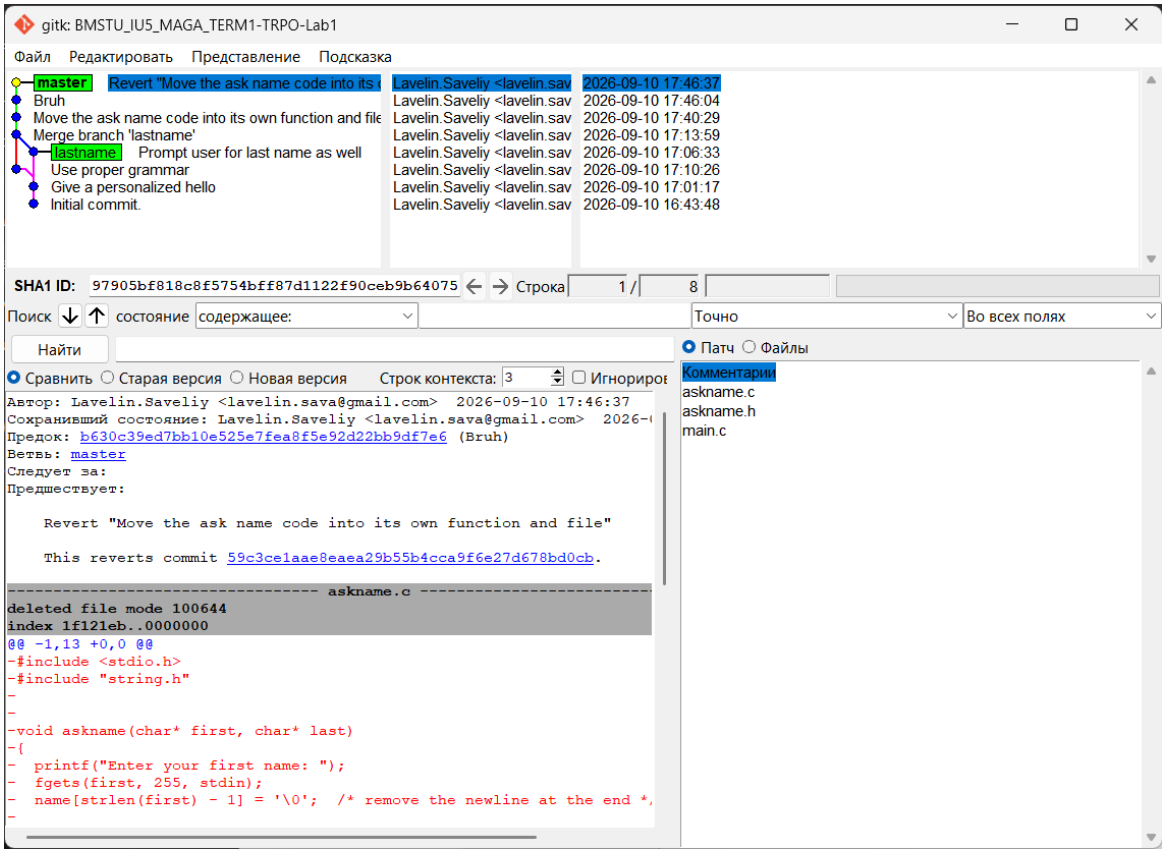


Рисунок 8 – Фиксация отката изменений (revert)

2.7 Публикация изменений на удалённом репозитории (GitHub)

Для совместной разработки локальный репозиторий опубликован на удалённом сервере GitHub: на сайте создан новый удалённый репозиторий, после чего с помощью команды push в него отправлены ветки локального репозитория. На рисунке 9 показано содержимое опубликованного репозитория на GitHub: список файлов проекта (main.c, askname.c, askname.h) с сообщениями и датами последних фиксаций для каждого файла.

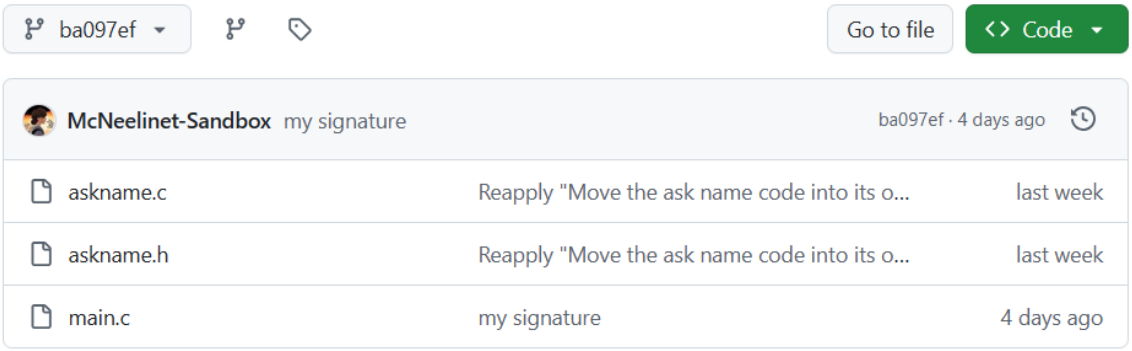


Рисунок 9 – Содержимое репозитория, опубликованного на GitHub

Далее продемонстрирована коллективная разработка через механизм форков и pull request’ов. Члены команды сделали форк репозитория и внесли в свою копию изменения. Был оформлен pull request на слияние. На рисунке 10 приведён список открытых pull request’ов репозитория на GitHub.

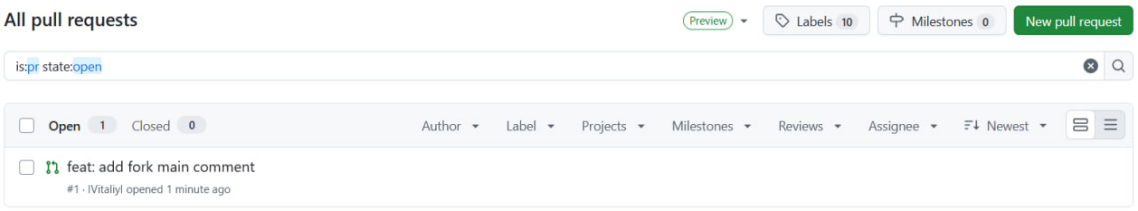


Рисунок 10 – Список открытых pull request’ов репозитория на GitHub

После проверки предложенных изменений pull request был принят и слит в ветку master. На рисунке 11 в окне gitk показана итоговая история удалённой ветки: слияние отображено отдельной фиксацией «Merge pull request #1 from IVitaliyl/master», а в нижней части окна приведён diff, демонстрирующий добавленную участником строку в файле main.c.

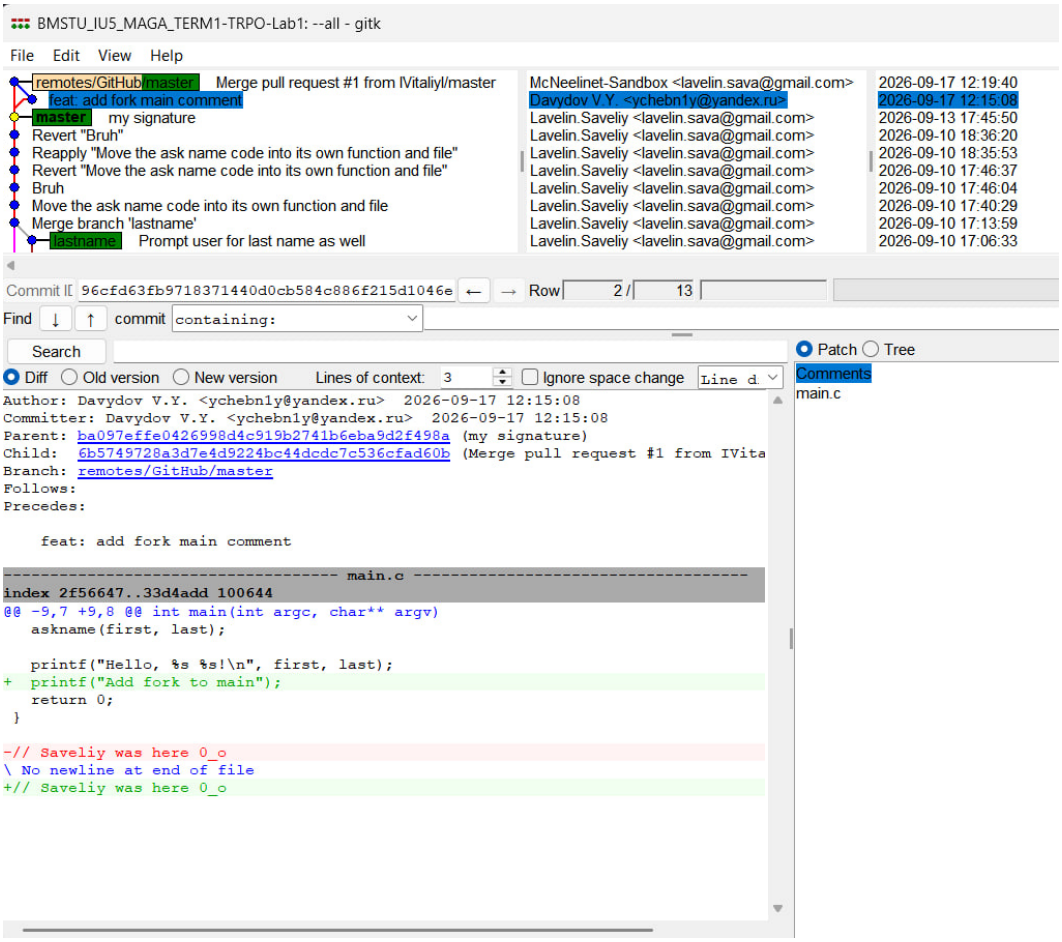


Рисунок 11 – История удалённой ветки после слияния pull request’a (gitk)

Таким образом, продемонстрирован полный цикл коллективной разработки через GitHub: публикация локального репозитория на удалённом сервере, внесение изменений сторонним участником через форк и pull request, проверка и слияние предложенных изменений владельцем репозитория, а также просмотр итоговой истории слияний и различий, внесённых разными участниками проекта.

3 Дополнительное задание

В рамках дополнительного задания продемонстрирована работа с тегами, командами отмены изменений revert и reset, перебазированием веток, временным сохранением незавершённых изменений и серией патчей.

3.1 Создание и применение тегов

Для фиксации состояния репозитория, соответствующего версии 1.0, создан аннотированный тег v1.0. Команде `git tag -a` переданы идентификатор коммита ba097ef и сообщение «Doing additional tasks». Наличие тега проверено командами `git tag` и `git log --oneline --decorate`. Как видно на рисунке 12, тег v1.0 указывает на выбранный коммит, не изменяя историю ветки master.

```
>git log --oneline
ba097ef (HEAD -> master) my signature
cd4bafa Revert "Bruh"
2c96f8f Reapply "Move the ask name code into its own function and file"
97905bf Revert "Move the ask name code into its own function and file"
b630c39 Bruh
59c3ce1 Move the ask name code into its own function and file
a04139a Merge branch 'lastname'
3d3b65c Use proper grammar
9b8a7ca (lastname) Prompt user for last name as well
3fdaf25 Give a personalized hello
1b4b40d Initial commit.

>git tag -a v1.0 ba097ef -m "Doing additional tasks"

>git tag
v1.0

>git log --oneline --decorate
ba097ef (HEAD -> master, tag: v1.0) my signature
cd4bafa Revert "Bruh"
2c96f8f Reapply "Move the ask name code into its own function and file"
97905bf Revert "Move the ask name code into its own function and file"
b630c39 Bruh
59c3ce1 Move the ask name code into its own function and file
a04139a Merge branch 'lastname'
3d3b65c Use proper grammar
9b8a7ca (lastname) Prompt user for last name as well
3fdaf25 Give a personalized hello
1b4b40d Initial commit.
```

Рисунок 12 – Создание аннотированного тега v1.0

3.2 Отмена изменений с помощью revert

После создания ошибочного коммита «Probably this is a mistake...» на него установлен тег v1.0.1.

Команда `git revert v1.0.1` отменила это изменение. Тег использован как ссылка на требуемый коммит. Git создал новый коммит, содержащий обратные изменения, при этом исходный ошибочный коммит остался в истории. Результат операции представлен на рисунке 13.

```
>git log --oneline --decorate
9d8f21b (HEAD -> master) Probably this is a mistake...
ba097ef (tag: v1.0) my signature
cd4bafa Revert "Bruh"
2c96f8f Reapply "Move the ask name code into its own function and file"
97905bf Revert "Move the ask name code into its own function and file"
b630c39 Bruh
59c3ce1 Move the ask name code into its own function and file
a04139a Merge branch 'lastname'
3d3b65c Use proper grammar
9b8a7ca (lastname) Prompt user for last name as well
3fdaf25 Give a personalized hello
1b4b40d Initial commit.

>git tag -a v1.0.1 9d8f21b -m "OH NO CRITICAL BUG"

>git revert v1.0.1
hint: Waiting for your editor to close the file... unix2dos: converting file C:/Users/McNeeli
net/Home/Programming/Repositories/BMSTU_IU5_MAGA_TERM1-TRPO-Lab1/.git/COMMIT_EDITMSG to DOS f
ormat...
dos2unix: converting file C:/Users/McNeelinet/Home/Programming/Repositories/BMSTU_IU5_MAGA_TE
RM1-TRPO-Lab1/.git/COMMIT_EDITMSG to Unix format...
[master 5b35633] Revert "Probably this is a mistake..."
 1 file changed, 1 insertion(+), 1 deletion(-)

>git log --oneline --decorate
5b35633 (HEAD -> master) Revert "Probably this is a mistake..."
9d8f21b (tag: v1.0.1) Probably this is a mistake...
ba097ef (tag: v1.0) my signature
cd4bafa Revert "Bruh"
2c96f8f Reapply "Move the ask name code into its own function and file"
97905bf Revert "Move the ask name code into its own function and file"
b630c39 Bruh
59c3ce1 Move the ask name code into its own function and file
a04139a Merge branch 'lastname'
3d3b65c Use proper grammar
9b8a7ca (lastname) Prompt user for last name as well
3fdaf25 Give a personalized hello
1b4b40d Initial commit.

>
```

Рисунок 13 – Отмена коммита командой revert с применением тега

3.3 Сброс состояния с помощью reset

Для возврата ветки master к ранее отмеченной версии выполнена команда `git reset --hard v1.0`. В отличие от revert, команда reset переместила указатель ветки непосредственно на коммит, отмеченный тегом v1.0, и привела индекс и рабочий каталог к его состоянию. Коммиты, созданные после версии 1.0, перестали входить в текущую историю ветки, что показано на рисунке 14.

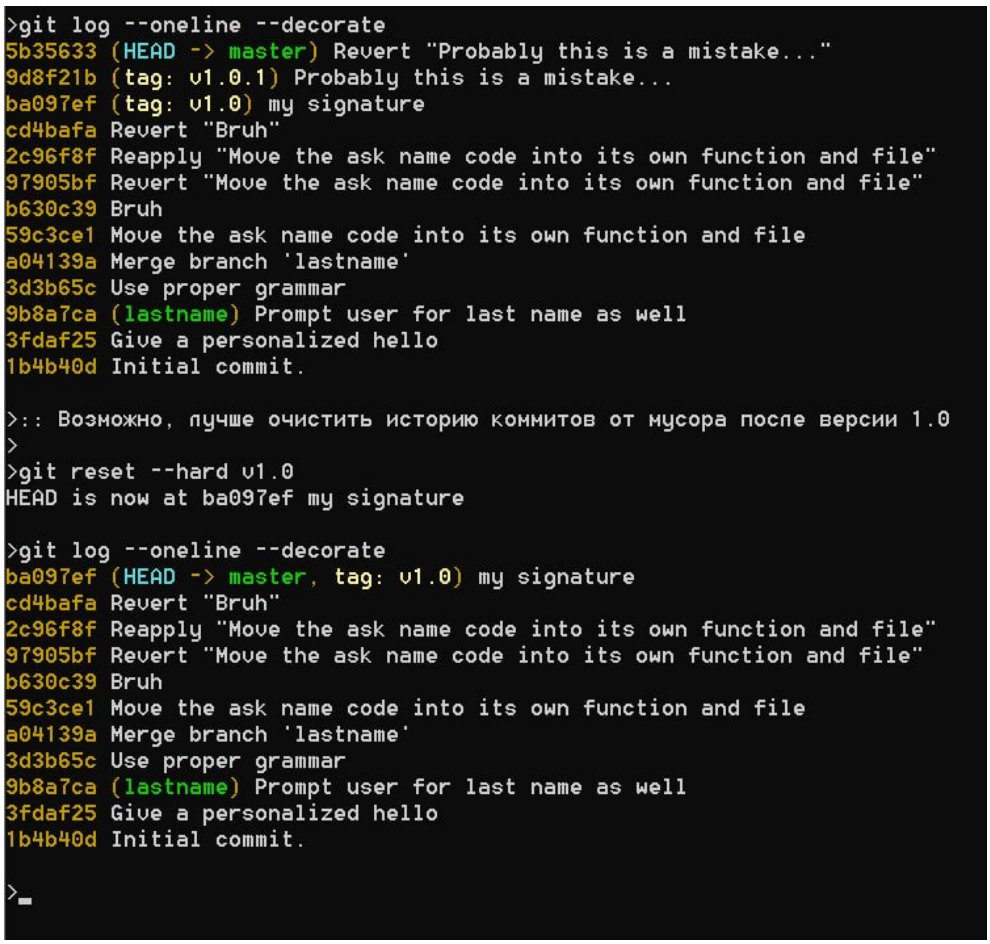


Рисунок 14 – Жёсткий сброс ветки master до состояния, отмеченного тегом v1.0

3.4 Перебазирование ветки (rebase)

Для демонстрации перебазирования создана ветка readme, в которой добавлен файл readme.md и выполнен коммит «Readme: added basic description». В ветке master независимо создан файл с тем же именем, но другим содержимым, после чего выполнено переключение обратно на ветку readme. Подготовка веток к перебазированию показана на рисунке 15.

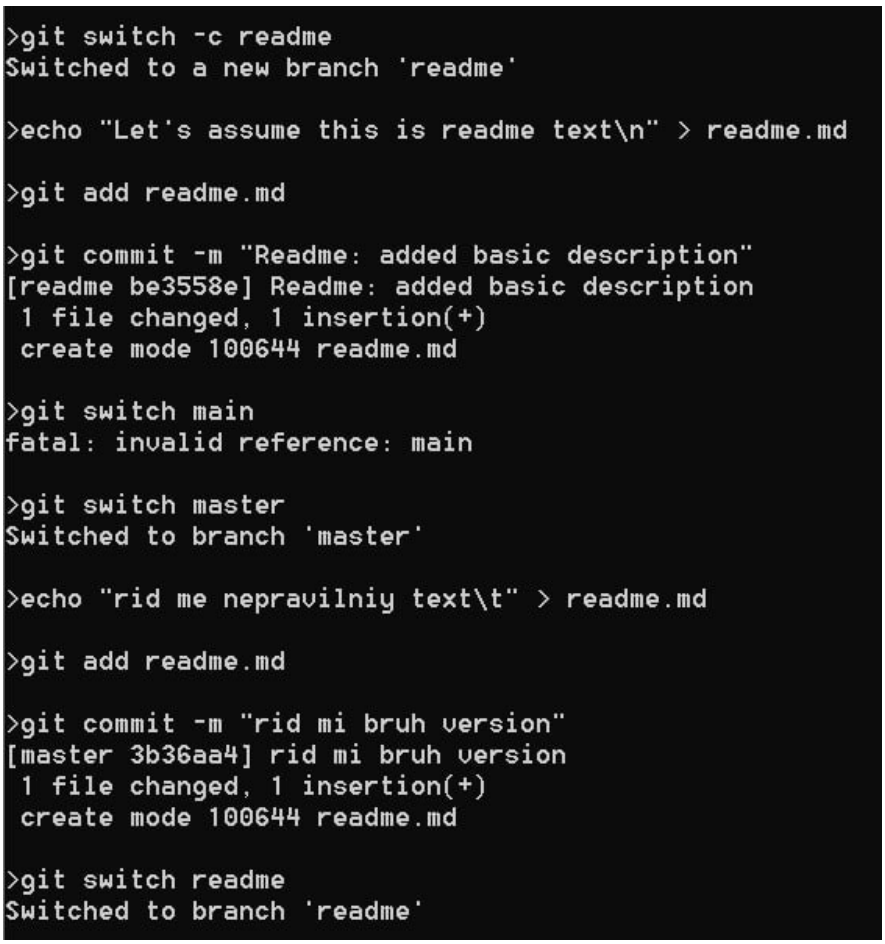


Рисунок 15 – Подготовка веток master и readme к перебазированию

При выполнении команды `git rebase master` возник конфликт добавления файла readme.md, так как файл был независимо создан в обеих ветках. Конфликт разрешён вручную, исправленный файл добавлен в индекс командой `git add readme.md`, после чего перебазирование продолжено командой `git rebase --continue`. Итоговый граф истории на рисунке 16 показывает, что коммит ветки readme был пересоздан поверх актуального состояния master.

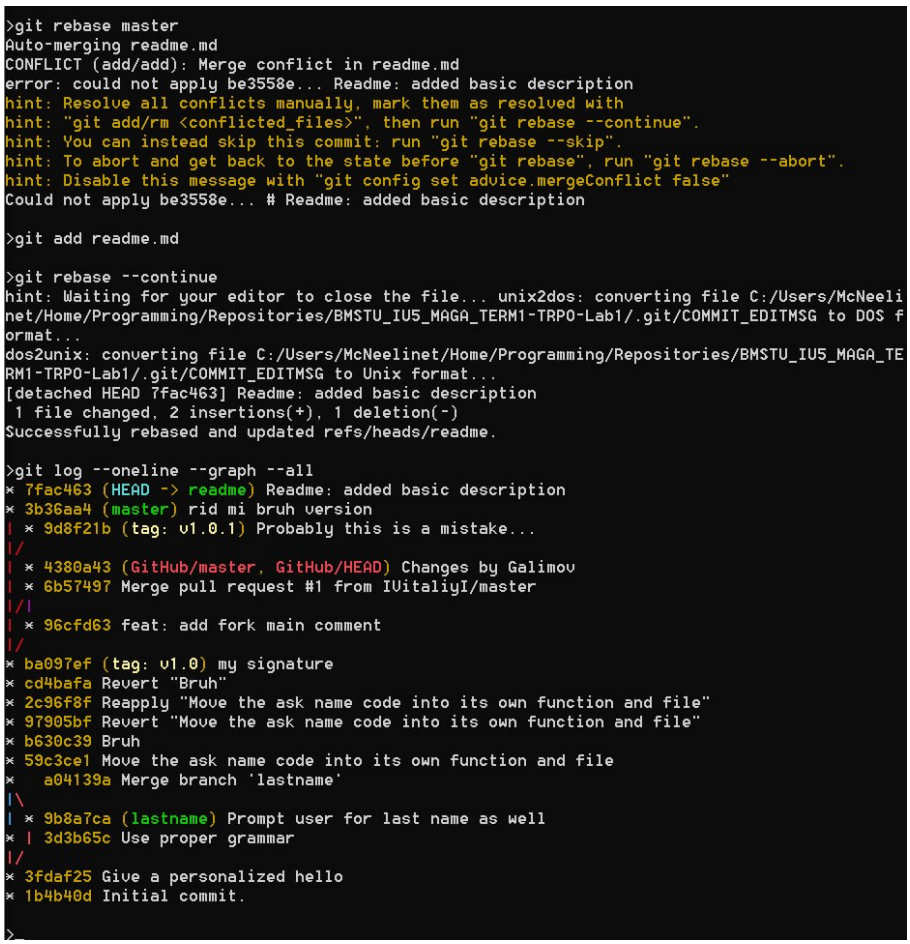


Рисунок 16 – Разрешение конфликта и завершение перебазирования ветки readme

3.5 Временное сохранение изменений (stash)

В файл readme.md внесено незавершённое изменение, после чего рабочее состояние сохранено командой `git stash push -m` с сообщением «WIP: новый крутой функционал». Команда `git stash list` подтвердила появление записи stash. После этого в файл askname.h внесено срочное исправление и создан отдельный коммит «hotfix». Команда `git stash pop` восстановила отложенное изменение readme.md и одновременно удалила использованную запись из stash. Затем восстановленное изменение было зафиксировано отдельным коммитом. Полная последовательность действий приведена на рисунке 17.

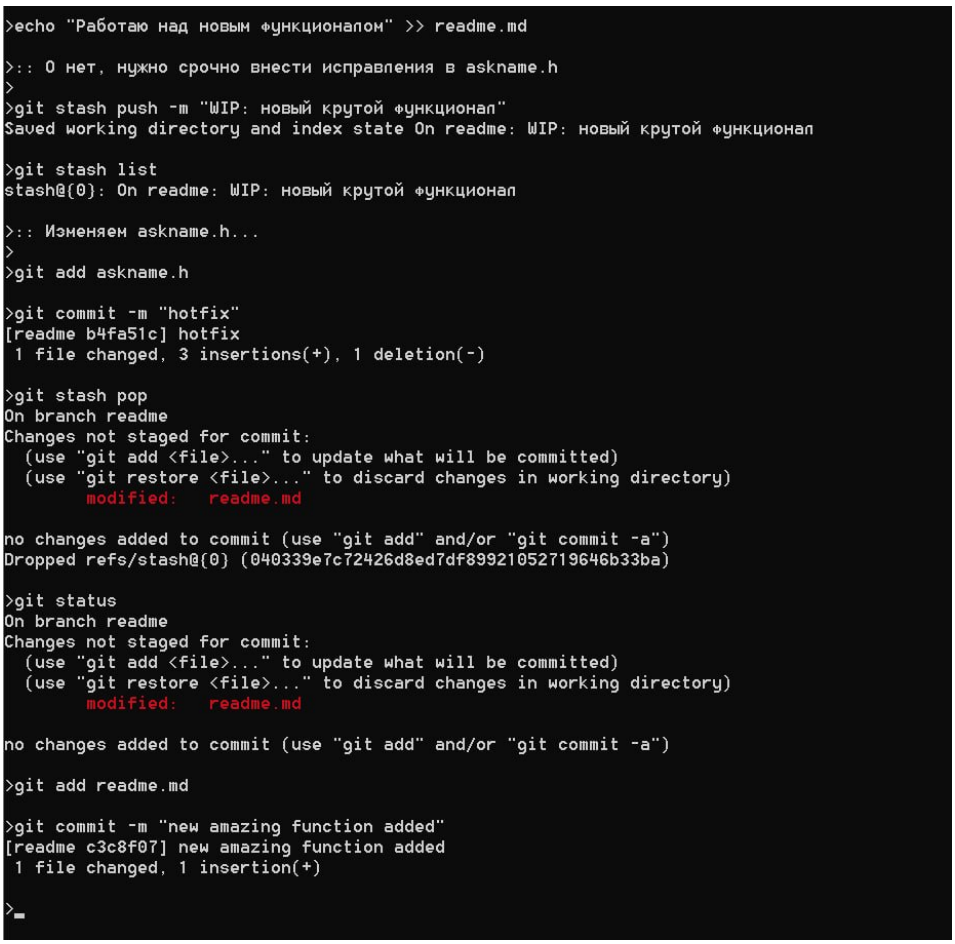


Рисунок 17 – Сохранение изменений в stash и их последующее восстановление

3.6 Создание и применение серии патчей

В каталоге patches подготовлена серия из трёх патчей, соответствующих последовательным изменениям «Feature part 1», «Feature part 2» и «Feature part 3». Каждый патч применён командой `git am`. Git сохранил авторство и сообщения исходных коммитов; выведенные предупреждения относятся к завершающим пробелам в строках патчей и не препятствуют их применению. Процесс показан на рисунке 18.

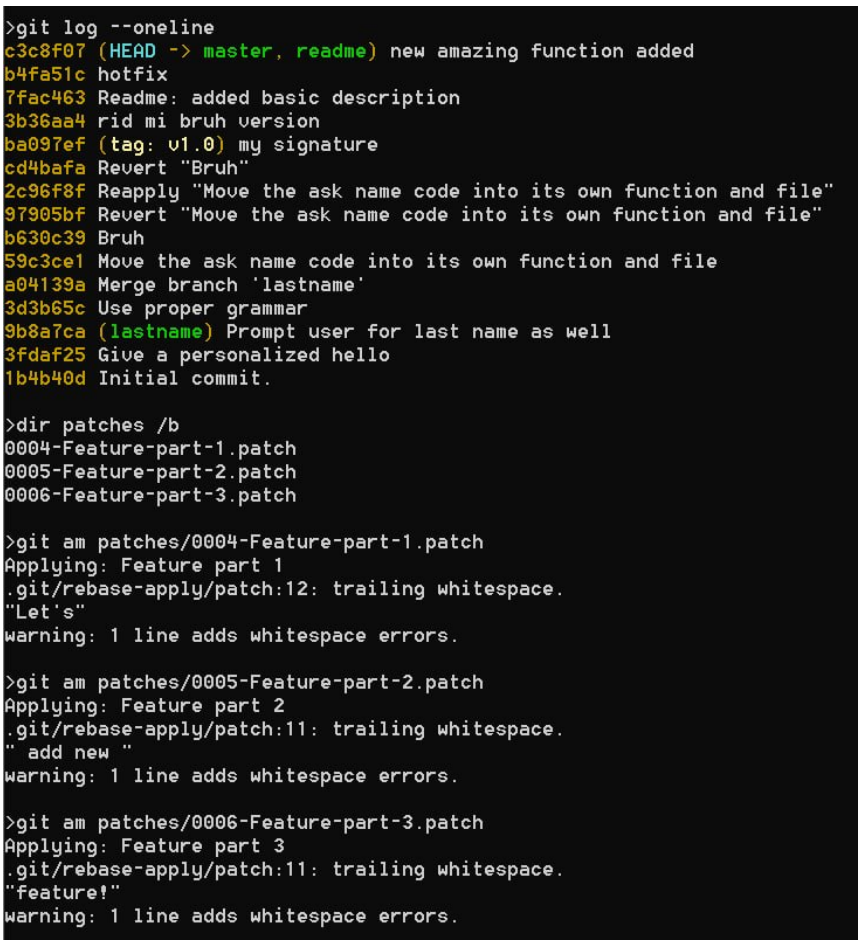


Рисунок 18 – Последовательное применение серии патчей командой git am

После применения патчей выполнена команда `git log --oneline`. В истории ветки master появились три последовательных коммита Feature part 1, Feature part 2 и Feature part 3, что подтверждает корректное применение всей серии. Итоговая история показана на рисунке 19.

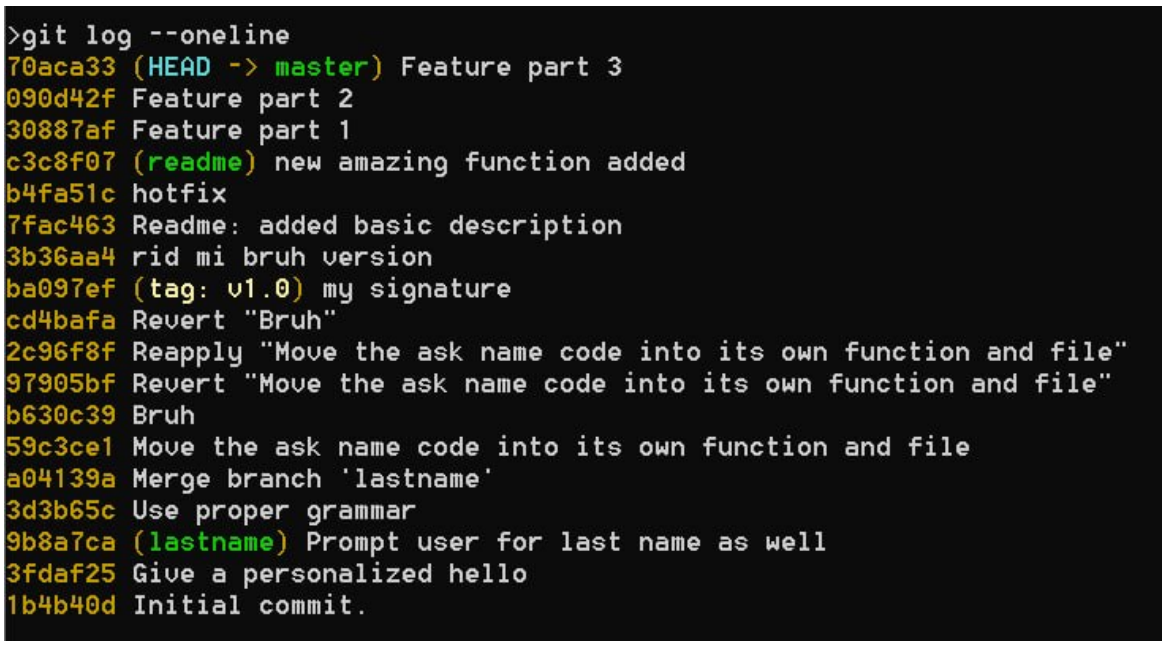


Рисунок 19 – История репозитория после применения серии патчей

Для создания серии использована команда `git format-patch` с параметрами `-6 -o patches`. В каталоге patches сформировано шесть файлов, по одному для каждого из шести последних коммитов. Имена файлов содержат порядковый номер и сообщение соответствующего коммита. Результат создания серии представлен на рисунке 20.



Рисунок 20 – Создание серии патчей командой git format-patch

Таким образом, в дополнительном задании продемонстрированы два способа отмены изменений, использование тегов как именованных указателей на коммиты, перенос коммитов между основаниями с помощью rebase, временное сохранение рабочего состояния в stash, а также перенос серии изменений посредством патчей.

4 Заключение

В ходе выполнения лабораторной работы изучены основные возможности системы контроля версий Git и приобретены практические навыки работы с ней через графический интерфейс Git GUI/gitk.

В рамках работы был создан локальный репозиторий, продемонстрирована фиксация изменений (в том числе выборочная – с отменой отдельных строк перед коммитом), создание и переключение веток, слияние веток с разрешением конфликта, просмотр истории изменений и авторства строк кода, а также отмена изменений через revert. Репозиторий опубликован на удалённом сервере GitHub, продемонстрирована синхронизация локальных и удалённых изменений.

Полученные навыки позволяют организовать коллективную разработку программного проекта с сохранением полной истории изменений и возможностью параллельной работы над разными частями функционала.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Словарь терминов для Git и GitHub // HTML Academy: [сайт]. – URL: https://htmlacademy.ru/blog/articles/git-and-github-glossary (дата обращения: 17.09.2026).

2. Git Wizardry // Habr: [сайт]. – URL: http://habrahabr.ru/post/60347/ (дата обращения: 17.09.2026).

3. Инструментальные средства управления версиями // КиберПедия: [сайт]. – URL: https://cyberpedia.su/11x9efe.html (дата обращения: 17.09.2026).

4. Git: наглядная справка // Mark Lodato's blog: [сайт]. – URL: http://marklodato.github.io/visual-git-guide/index-ru.html (дата обращения: 17.09.2026).

5. Основы работы с Git // Calculate Linux: [сайт]. – URL: http://www.calculate-linux.ru/main/ru/git (дата обращения: 17.09.2026).

6. Git Workflow // Habr: [сайт]. – URL: http://habrahabr.ru/post/60030/ (дата обращения: 17.09.2026).

7. Gitflow Workflow // Atlassian Git Tutorial: [сайт]. – URL: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow (дата обращения: 17.09.2026).

8. Виноградова М.В., Авдеев Ю.В., Ваняшкин Ю.Ю., Мурашко И.А., Оганесян Р.Р., Пинская Н.М. Методические указания к лабораторным работам по курсу «Технологии разработки программного обеспечения». Лабораторная работа №2 «Управление версиями и коллективной разработкой программного проекта на примере утилиты Git». – М.: МГТУ им. Н.Э. Баумана, 2022.